

RAG-KnowledgeNexus DB Design Guide

RAG-KnowledgeNexus — 数据库设计使用指南

> 企业智能文档问答助手 • 基于 Qdrant 的数据库设计与开发指南

交付物总览

数据库结构（12张表，5层架构）

第一层：用户与权限体系（3 张表）

- ☐ sys_dept: 部门表（树形结构，parent_id 自关联）
- ☐ sys_user: 用户表（role: 0=超管，1=KB管理员，2=普通员工）
- ☐ sys_operation_log: 操作日志表（审计追踪）

关键字段：

- ☐ `sys\_user.role`: 0=超管（全权限），1=KB管理员（管理指定KB），2=普通员工（仅查询）
- ☐ `sys\_dept.parent\_id`: 树形结构，支持多级部门

第二层：知识库与文档体系（3 张表）— RAG 数据基础

- ☐ doc_knowledge_base: 知识库表（visibility: 0=私有，1=部门，2=全员）
- ☐ doc_document: 文档表（PDF/DOCX/TXT 上传，parse_status 追踪解析进度）
- ☐ doc_chunk: 分片表【核心表】（content + vector_id + embed_status）

关键字段：

- ☐ `doc\_chunk.vector\_id`: Qdrant Point ID，关联向量库中的记录
- ☐ `doc\_chunk.embed\_status`: 0=待向量化，1=完成，2=失败
- ☐ `doc\_chunk.kb\_id`: 冗余字段，避免多表 JOIN，加速权限过滤

第三层：问答与对话体系（3 张表）

- ☐ qa_conversation: 对话会话表
- ☐ qa_message: 问答消息表【核心表】（question/answer/source_chunks/quality_score）
- ☐ qa_feedback: 答案反馈表（赞/踩，评估 RAG 质量）

第四层：工单管理体系（1 张表）

- ☐ ticket_order: 咨询工单表（AI 无法回答时自动转人工）

状态流转：0（待处理）→ 1（处理中）→ 2（已解决）→ 3（关闭）

第五层：数据统计体系（2 张表）

- ☐ stat_daily: 每日统计汇总（定时任务每天凌晨计算）
- ☐ stat_hot_question: 热门问题统计（MD5 去重）

核心设计思路

1. 为什么在 doc_chunk 冗余 kb_id?

差的方案（3 表 JOIN）：

好的方案（单表查询，冗余 kb_id）：

冗余 kb_id 是安全的——分片一旦创建，所属知识库关系就固定了。

2. 为什么 qa_message.source_chunks 用 JSON?

JSON 格式灵活，可存储 `[[chunk_id, doc_id, doc_title, score]]`，一次查询完成。

3. 权限隔离如何设计?

知识库有三种可见范围，在数据库层完成权限过滤（WHERE 条件）：

Qdrant 搜索时也带上 kb_id 作为 payload 过滤条件。

快速开始

1. 初始化数据库

2. 启动 Qdrant

3. 验证连接

Qdrant 向量库映射

RAG 系统使用 Qdrant 作为向量数据库。MySQL doc_chunk 表与 Qdrant Collection 的映射关系：

检索流程

1. 用户问题经 DashScope text-embedding-v3 转为 1024 维向量
2. 在 Qdrant 中 ANN 搜索（带 kb_id 和 is_deleted=0 过滤条件）
3. 得到匹配的 Point ID 列表（= doc_chunk.id）
4. 通过 chunk ID 反查 MySQL doc_chunk 获取完整文本
5. 与 BM25 关键词检索结果做 RRF 融合排序
6. 拼接 Prompt 发送给 LLM（阿里云 Qwen）

Collection 配置建议

混合检索（Hybrid Search）

不要只用向量检索。加上 BM25 关键词检索（MySQL FULLTEXT），两路结果用 RRF 算法融合排序：

为什么好？纯向量检索对精确词汇（如产品型号 XR-7800-Pro）效果差，混合检索是工业界标准做法。

Qdrant 选型理由

结论：Qdrant 是中间最优解——真正的向量数据库，单二进制部署，运维成本与 Lucene 相当，性能与专业度远超 Lucene。

常见问题解答

Q1: 如何保证 Qdrant 和 MySQL 的数据一致性?

两阶段策略: 先在 MySQL 创建 doc_chunk 记录 (embed_status=0), 异步向量化后调用 Qdrant REST API upsert, 成功后设置 embed_status=1 并填写 vector_id。如果 Qdrant 写入失败, embed_status 保持 0 (待向量化), 系统可定期重试。

对于删除操作: 软删除 doc_chunk 记录后, 异步调用 Qdrant 的 delete_points 接口清理对应 Point。

Q2: Qdrant Payload 过滤和 Lucene 二次过滤有什么区别?

Lucene 向量检索不支持标量过滤, 必须先检索全部向量再在应用层做二次过滤。Qdrant 的 Payload 过滤是原生的——ANN 搜索时直接带上过滤条件, 搜索引擎内部完成过滤, 减少不必要的数据传输和计算。

Q3: 数据量增长时如何扩展?

Qdrant 单机可支撑百万级 1024 维向量。如果数据量超出单机能力, Qdrant 支持分布式集群部署 (通过 REST API 透明扩展)。

索引与性能优化建议

Qdrant 性能优化

- ☐ 预创建 Collection, 指定 vector_size=1024, distance=Cosine
- ☐ 对高频过滤字段 kb_id、doc_id 创建 Payload 索引
- ☐ 每批 25 条调用 Qdrant upsert, 减少 HTTP 请求次数
- ☐ HNSW 参数: m=16, ef_construct=100

MySQL 索引优化

- ☐ 全文索引 ft_content 使用 ngram 解析器, 支持中文 BM25 检索
- ☐ 组合索引 idx_kb_status_id (kb_id, embed_status, id) 覆盖分片查询
- ☐ 高频查询尽量使用索引覆盖, 避免回表查询

开发指引

核心代码结构

QdrantIndexService 核心接口

关键索引列表

外键约束总览

删除策略说明: CASCADE 级联删除, SET NULL 置空删除, RESTRICT 拒绝删除。

下一步

1. 导入 SQL 到 MySQL
2. 启动 Qdrant 并创建 Collection

3. 实现 QdrantIndexService REST API
4. 实现向量化 + Qdrant 入库链路
5. 实现混合检索（向量 + BM25 + RRF）RAG 核心链路

版本：v1.0

文档更新：2025-06-11

作者：个人项目